

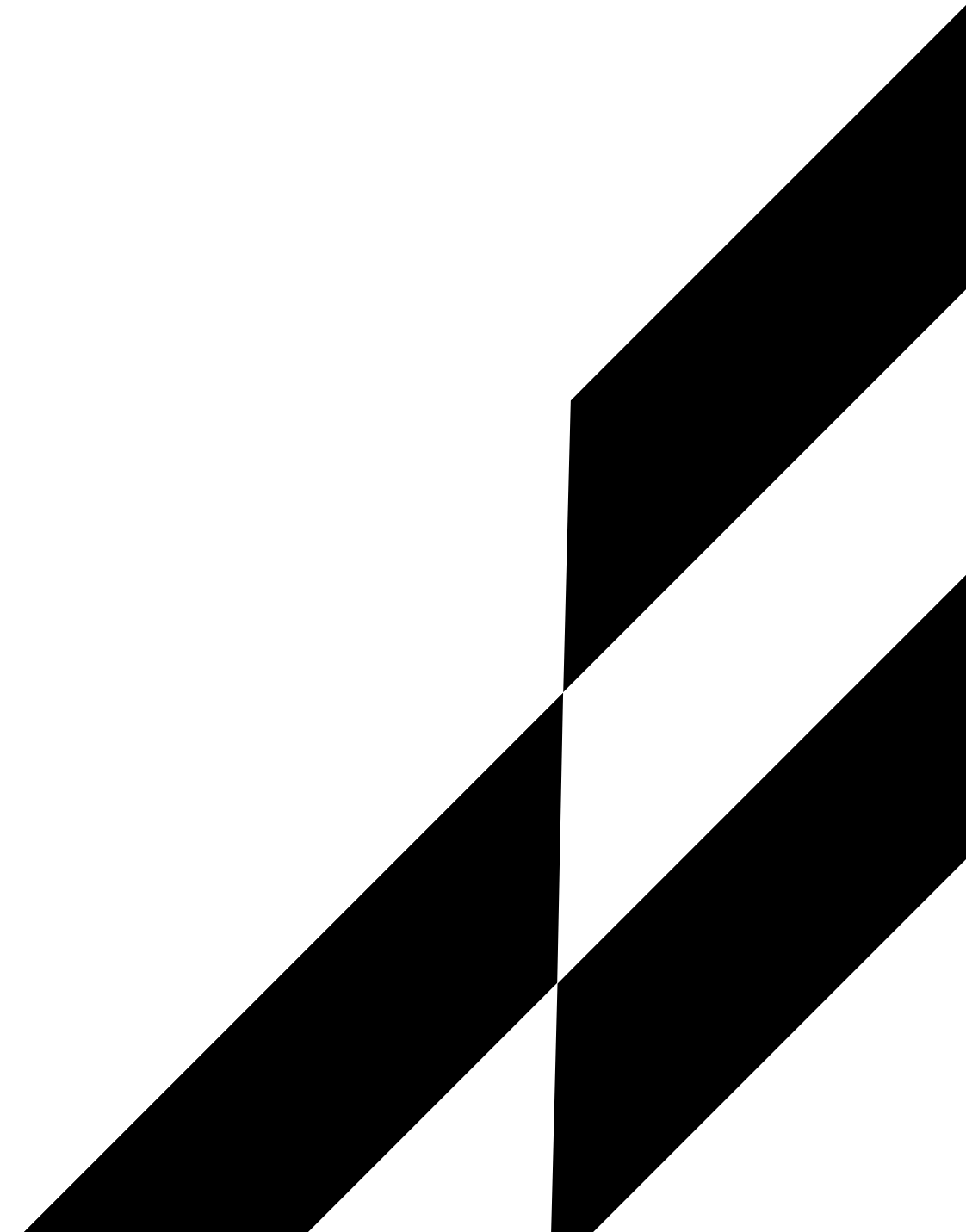
Semana 2

Python para Ingeniería de Datos

Bootcamp de Data Engineering – Ian Saura

Objetivo principal: Dominar los fundamentos de Python aplicados a la construcción de pipelines de datos.

Contenido clave: Estructuras de datos, lectura de archivos, pandas, transformaciones y automatización básica.



Repaso

Semana Anterior

Resumen de aprendizajes clave

¿Qué es un Data Engineer?
Profesional que diseña, construye y mantiene sistemas de datos escalables para análisis y reporting.

Etapas del pipeline: Ingesta, procesamiento, almacenamiento, modelado y exposición de datos.

Entrega de ejercicios: Discusión de soluciones enviadas y aprendizajes comunes sobre manejo de datos.

¿Por qué Python en Data Engineering?

Lenguaje clave para procesamiento de datos

Popularidad y comunidad: Python es uno de los lenguajes más usados en ciencia de datos e ingeniería, con vasta documentación y soporte.

Versatilidad: Se aplica en scraping, automatización, procesamiento de archivos y comunicación con APIs.

Ecosistema rico: Ampla disponibilidad de librerías como pandas, requests, numpy y airflow.

Herramientas que Usaremos Hoy

Entorno de trabajo en Python para ingeniería de datos

- Jupyter / Colab: Entornos interactivos ideales para experimentar con código y documentar el proceso.
 - Pandas: Librería clave para manipulación eficiente de datos tabulares con estructuras tipo DataFrame.
 - Módulos built-in: Uso de csv, json, os, datetime para tareas comunes sin necesidad de instalar librerías externas.
-

Tipos de Datos y Estructuras Básicas

Fundamentos esenciales de Python

Tipos primitivos: Python trabaja con int, float, str y bool como base para todos los datos.

Estructuras compuestas: Uso de listas, diccionarios, tuplas y sets para organizar información compleja.

Inmutabilidad y mutabilidad: Comprender qué estructuras pueden modificarse y cuáles no impacta en el diseño del código.

Listas y Diccionarios

Manipulación básica con estructuras clave

Operaciones con listas

Crear, acceder y modificar listas utilizando índices y slices.

Iteraciones comunes

Uso de for, if, in, enumerate e items() para recorrer listas y diccionarios.

Diccionarios en acción

Uso de claves y valores para almacenar y acceder a información estructurada.

Actividad 1

A partir de este diccionario de ventas:

```
ventas = {  
    "Juan": 150,  
    "Ana": 200,  
    "Luis": 350,  
    "Ana": 50  
}
```

Convertirlo en una lista de tuplas (cliente, total).
¿Cómo sumar los valores de clientes repetidos?

Funciones Útiles para Transformar Datos



zip()

Combina elementos de múltiples listas en tuplas; útil para relacionar datos paralelos.



map() y filter()

Permiten aplicar funciones o filtrar elementos sin bucles explícitos.



lambda

Funciones anónimas para aplicar lógica rápida y concisa en transformaciones.

Módulo CSV y Lectura Manual

Procesamiento de archivos paso a paso

01

Usar `open()` para leer archivos CSV como texto, permitiendo un control granular.

02

Uso de `split()`
Separar columnas usando delimitadores personalizados como la coma o punto y coma.

03

Casting de tipos
Convertir valores a `int`, `float` o `bool` para procesarlos correctamente.

Actividad 2

Archivo: ventas.csv

producto,precio,cantidad

Manzana,100,3

Banana,50,5

Pera,80,2

- Abrir el archivo y leerlo línea por línea.
- Separar cada línea por comas.
- Imprimir producto y total (precio × cantidad)

JSON y Requests

Consumo y manipulación de datos estructurados

- Leer JSON desde archivo: Usar json.load() para interpretar archivos como estructuras de diccionario.
- APIs públicas: Acceder a datos externos con requests.get() y convertir respuestas en JSON.
- Diccionesarios anidados: Explorar estructuras complejas con múltiples niveles de claves y listas.

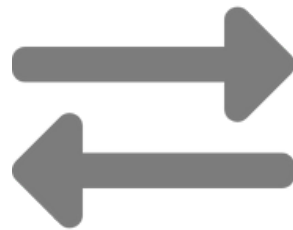
Introducción a pandas

**Estructura central para
datos tabulares en
Python**



¿Qué es un DataFrame?

Estructura bidimensional similar a una hoja de cálculo o tabla de base de datos.



Comparación con Excel y SQL

Ofrece lo mejor de ambos mundos: flexibilidad como Excel y eficiencia como SQL.



Ventajas

Filtrado eficiente, operaciones vectorizadas, limpieza de datos simplificada.

Cargar un CSV con pandas

Exploración inicial de datos tabulares **Lectura con `pd.read_csv()`**

Carga rápida de archivos CSV en DataFrames con manejo automático de tipos.

Exploración con `head()`, `info()`, `shape`

Visualiza las primeras filas, tipos de datos y dimensiones del dataset.

Primer diagnóstico

Identificar columnas relevantes, nulos y posibles inconsistencias.

Actividad 3

Archivo: ventas.csv

- Cargar el CSV con `pd.read_csv()`
- Mostrar las primeras filas con `head()`
- Ver información general con `info()` y forma con `shape`
- Contar cuántos productos hay y cuántas filas en total.

Seleccionar y Filtrar Columnas

Acceso eficiente a subconjuntos de datos

- Acceso a columnas: Seleccionar columnas individuales o múltiples por nombre.
 - Filtrado con `loc[]` e `iloc[]`: Acceder a filas y columnas usando etiquetas o índices numéricos.
 - Condiciones booleanas: Filtrar filas que cumplen criterios lógicos para segmentar datos.
-

Actividad 4

Archivo: ventas.csv

- Seleccionar solo las columnas producto y precio.
- Filtrar las filas donde precio sea mayor a 100.
- Mostrar solo los 10 primeros resultados filtrados.

Transformaciones Básicas

Limpieza y ajuste de datos con pandas

- Renombrar columnas: Usar `rename()` para actualizar nombres a formatos consistentes.
- Crear nuevas columnas: Generar variables a partir de operaciones sobre columnas existentes.
- Manejo de nulos: Eliminar con `dropna()` o rellenar con `fillna()` según el contexto.

Actividad 5

Archivo: ventas.csv

- Crear una nueva columna llamada ingresos = $\text{precio} \times \text{cantidad}$
- Cambiar el nombre de la columna producto a nombre_producto
- Reemplazar los valores nulos en categoria por "Sin categoría"

Agrupar y Resumir

Síntesis de datos con groupby y conteo

`groupby()`: Agrupa filas por categorías y permite aplicar funciones agregadas como `sum` o `mean`.

Sumar valores: Ejemplo: sumar ventas por categoría o calcular ingresos totales por producto.

Contar ocurrencias: Usar `value_counts()` para contar valores únicos y su frecuencia.

Actividad 6

Archivo: ventas.csv

- Agrupar por categoría y sumar la columna ingresos.
- Mostrar el resultado ordenado de mayor a menor.
- Contar cuántos productos distintos hay en cada categoría.

Guardar Resultados

Exportar datos limpios para su reutilización

- `to_csv()` y `to_json()`: Guardar DataFrames en formatos estándar para compartir o reutilizar.

- Evitar sobrescribir: Verificar nombres de archivo o agregar timestamps para evitar pérdida de datos.

- Separadores personalizados: Cambiar delimitador en CSV para compatibilidad con otras herramientas.

Flujo Típico de Limpieza

Checklist esencial para trabajar con datos

01

Cargar

Importar datos desde CSV, JSON u otros formatos usando pandas

02

Explorar

Revisar estructura, valores nulos y tipos de datos con `info()`, `head()`, `describe()`

03

Limpiar y transformar

Corregir nombres, crear columnas, eliminar o rellenar nulos

04

Guardar

Exportar los datos limpios para su uso futuro o en pipelines

Ejemplo Práctico

Aplicando lo aprendido a un caso real



Leer CSV de ventas

Importar datos simulados de productos y transacciones.



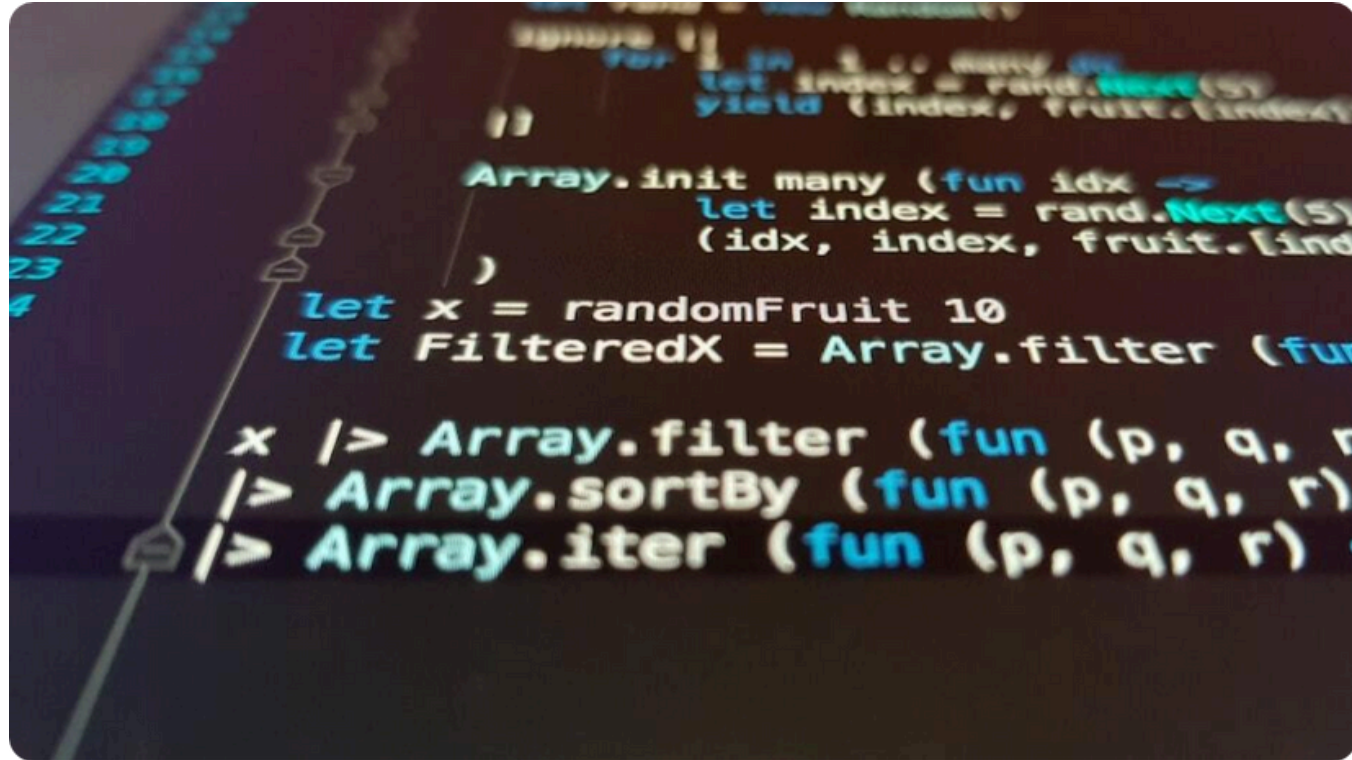
Limpiar columnas

Corregir nombres, tipos y valores faltantes en el dataset.



Sumar ventas y guardar

Agrupar por producto y exportar CSV limpio con ingresos totales.



Tips para Debugging y Performance

Optimización de código en Python

- Uso de `print()` y `type()`: Diagnosticar errores revisando tipos de datos y valores intermedios.
- Manejo de errores: Evitar interrupciones usando `try-except` y validaciones previas.
- Evitar loops innecesarios: Usar operaciones vectorizadas y aplicar funciones en lugar de iteraciones manuales.

Recapitulación



Fundamentos de Python

Estructuras de datos, lectura de archivos, funciones y módulos clave.



Uso de pandas

Manipulación y transformación de datos tabulares con eficiencia.



Aplicación a pipelines

Ejercicios prácticos que simulan pasos reales en ingeniería de datos.

¡Hora de practicar lo aprendido!

Ejercicios



Preguntas

Momento para resolver dudas y profundizar conceptos complejos.



Ejercicio 1: CSV

Limpiar, transformar y agrupar datos de un archivo de ventas.



Ejercicio 2 y 3

Trabajar con diccionarios, JSON y consumir APIs públicas.